



Permissões e Transações no MySQL

Compreender o papel das permissões e das transações no gerenciamento de dados é fundamental para garantir a segurança e integridade dos bancos de dados MySQL.

SM por **Salvador Melo**



Permissões (GRANT / REVOKE)

O que são permissões?

Permissões são direitos concedidos a usuários do banco que definem o que eles podem ou não fazer. Com elas, um administrador pode controlar com precisão quem pode ler, inserir, alterar ou excluir informações, garantindo segurança, integridade e conformidade dos dados.

Exemplo prático

No VendaIngressosDB, um usuário do tipo vendedor só deve poder visualizar eventos e registrar vendas, mas não deve poder visualizar ou editar os dados financeiros.

Isso é feito com comandos GRANT e REVOKE, criando limites de acesso por perfil de usuário.

Sintaxe de Permissões

Para gerenciar acesso no MySQL, primeiro criamos usuários e depois atribuímos permissões específicas.

Criação de Usuário

O comando CREATE USER cria contas de acesso ao servidor, mas inicialmente sem permissões.

Definição de Acesso

Especificamos quem pode se conectar e com qual senha, mas não o que pode fazer.

Sintaxe Básica

```
CREATE USER  
'nome_usuario'@'origem'  
IDENTIFIED BY 'senha';
```

Atribuição de Permissões

Depois da criação, usamos GRANT para definir o que cada usuário pode fazer.

Exemplos

-- 1. CREATE USER

-- Cria um novo usuário no MySQL:

```
CREATE USER 'fulano_de_tal'@'127.0.0.1' IDENTIFIED BY 'senha123456';
```

```
CREATE USER 'beltrano_de_tal'@'127.0.0.1' IDENTIFIED BY 'senha123456';
```

-- 2. Excluir user

```
DROP USER 'beltrano_de_tal'@'127.0.0.1';
```





Usuários existentes

-- 3. usuário existentes

```
select User from mysql.user;
```

```
SELECT USER();
```

Trocar senha

ALTER USER 'fulano_de_tal'@'127.0.0.1' IDENTIFIED BY '123456';

- Interpreta '123456' como senha em texto plano.
- Internamente, a senha será hasheada (ex: SHA256, caching_sha2, etc).
- Mais moderno, compatível com MySQL ≥ 5.7 e MariaDB $\geq 10.2.0$.
- Suporta outras opções como IDENTIFIED VIA para especificar o plugin de autenticação.

SET PASSWORD for 'fulano_de_tal'@'127.0.0.1' = PASSWORD('teste@senha');

- obsoleto: Em **versões mais recentes do MySQL/MariaDB**, o comando **pode aceitar senha em texto plano**
- **No MySQL ≥ 8.0 :** SET PASSWORD foi reimplementado para aceitar senha pura.

SET PASSWORD FOR 'fulano_de_tal'@'127.0.0.1' = 'teste@senha';

- **Erro:**  Isso acontece porque o MySQL/MariaDB está esperando o hash da senha — não o texto puro.
- Em versões antigas, **esperava o valor já hasheado** (como o erro que você viu: “should be a 41-digit hexadecimal”).

Mudar o user

Sintaxe básica do RENAME USER

```
RENAME USER 'usuario_antigo'@'host_antigo' TO 'usuario_novo'@'host_novo';
```



Exemplos práticos:

```
RENAME USER 'fulano_de_tal'@'127.0.0.1' TO 'ciclano'@'localhost';
```

Ou apenas mudando o nome, mantendo o host:

```
RENAME USER 'fulano_de_tal'@'127.0.0.1' TO 'ciclano'@'127.0.0.1';
```

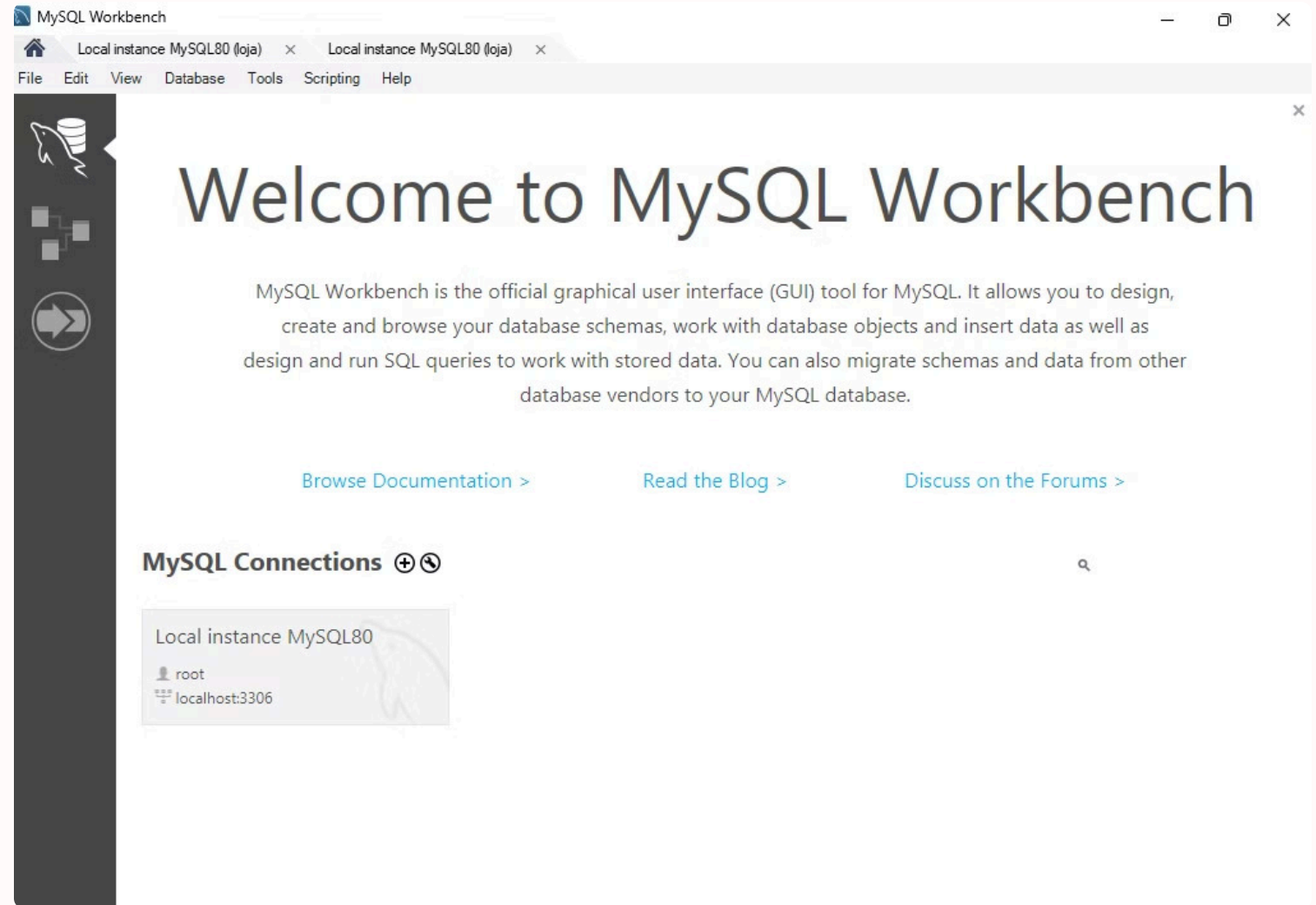
Ou apenas mudando o host:

```
RENAME USER 'fulano_de_tal'@'localhost' TO 'fulano_de_tal'@'127.0.0.1';
```

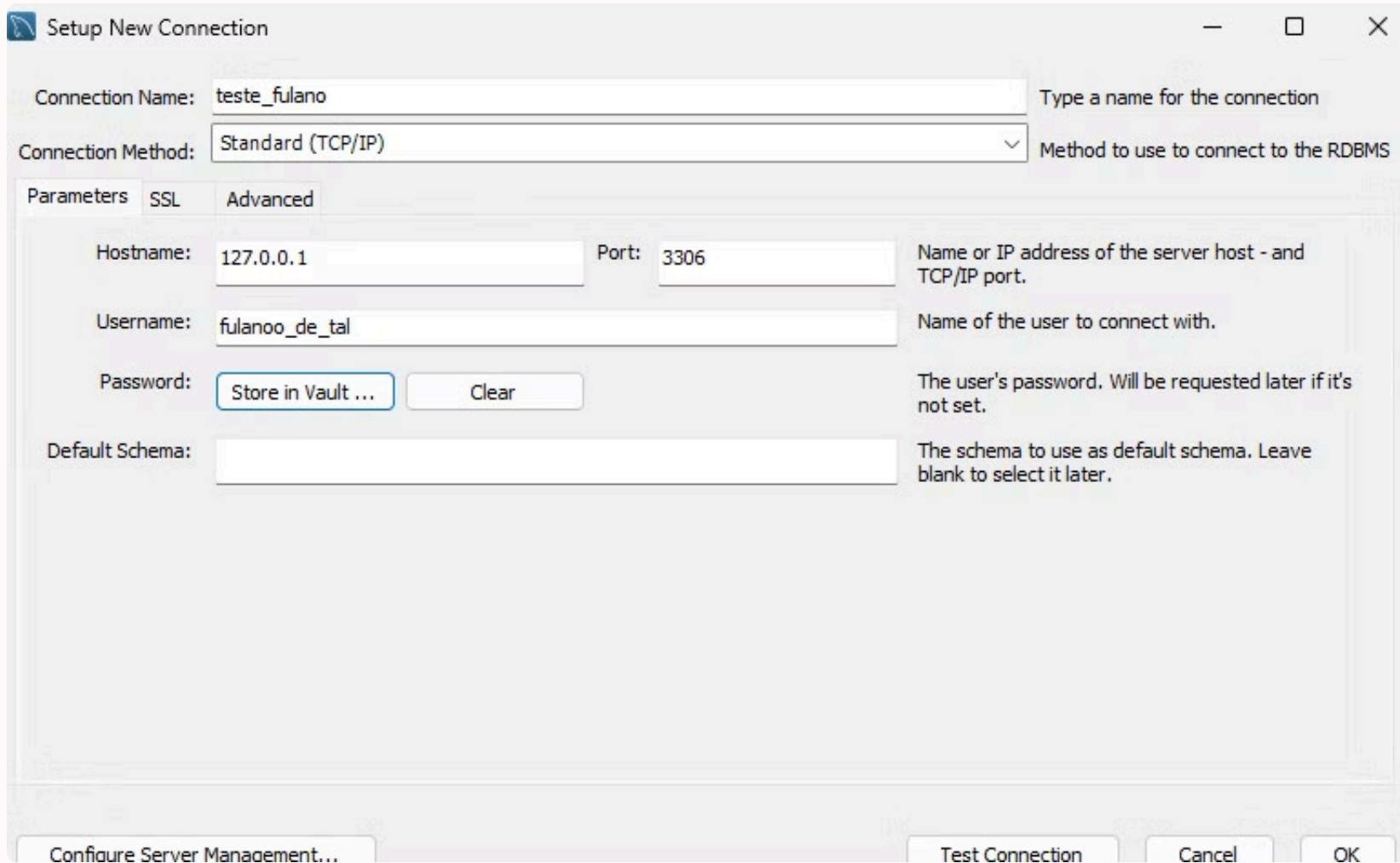
Cuidados e dicas

1. **Permissão necessária:** O usuário que executa o `RENAME USER` precisa ter **privilegio global** `UPDATE` na tabela `mysql.user` ou o **privilegio** `CREATE USER`.
2. **Senhas e privilégios são preservados:** Quando você renomeia um usuário, **os privilégios e senha são mantidos** no novo nome.
3. **Usuários com o mesmo nome mas host diferente são distintos:**
 - `'admin'@'localhost' ≠ 'admin'@'%'`
 - Por isso, o `RENAME` também pode mudar o host (muito útil).

Criando Nova Conexão



Criando Nova Conexão



Setup New Connection

Connection Name: teste_fulano Type a name for the connection

Connection Method: Standard (TCP/IP) Method to use to connect to the RDBMS

Parameters SSL Advanced

Hostname: 127.0.0.1 Port: 3306 Name or IP address of the server host - and TCP/IP port.

Username: fulanoo_de_tal Name of the user to connect with.

Password: The user's password. Will be requested later if it's not set.

Default Schema: The schema to use as default schema. Leave blank to select it later.

Store in Vault ... Clear

Configure Server Management... Test Connection Cancel OK

Criando Nova Conexão

MySQL Connections

Local instance MySQL80

 root

 localhost:3306

teste_fulano

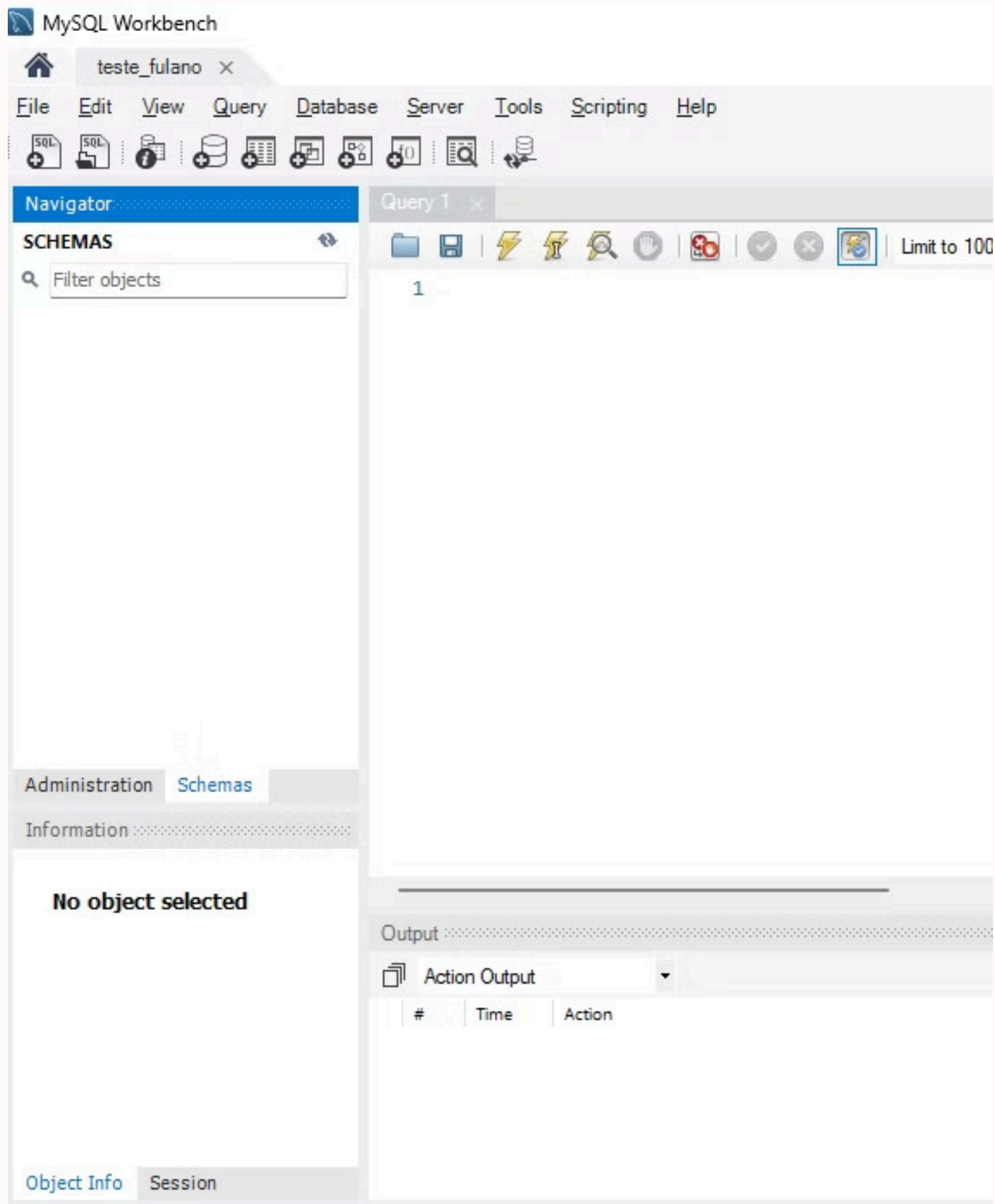
 fulano_de_tal

 127.0.0.1:3306

User: fulano_de_tal

Senha: SenhaForte@2025

Criando Nova Conexão



O que acontece se tentar usar sem GRANT?

Se o `analista_dados` tentar executar algo como `SELECT * FROM loja.produtos`, vai receber erro de acesso negado.

Concedendo Permissões (GRANT)

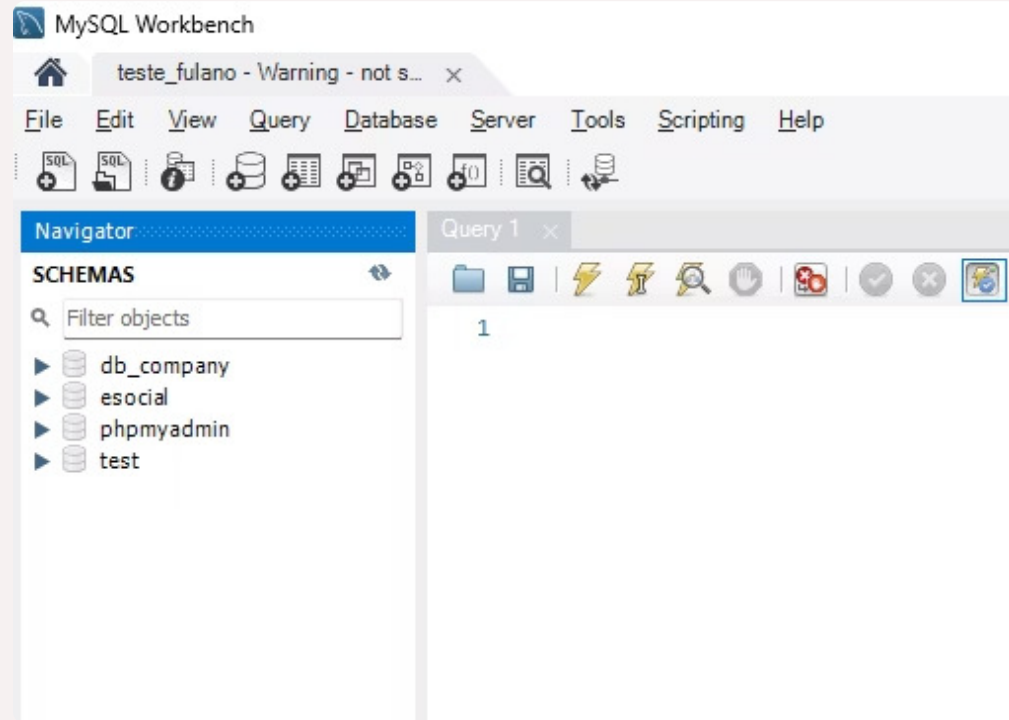
```
GRANT ALL PRIVILEGES ON *.* TO 'fulano_de_tal'@'127.0.0.1';
```

Explicando por partes:

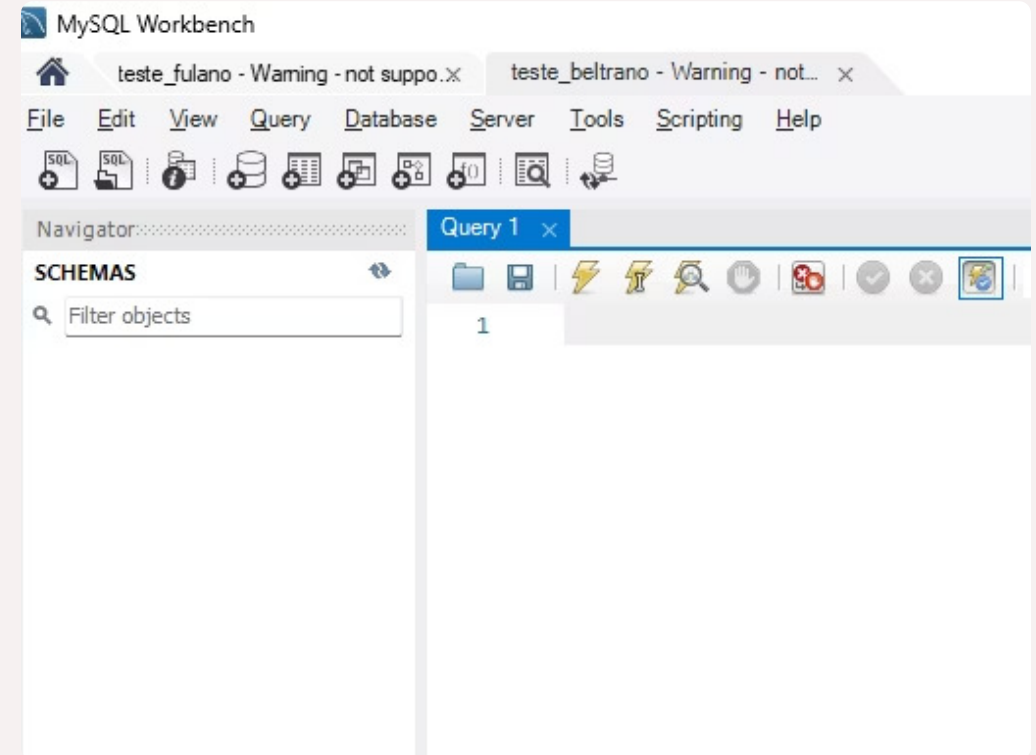
1. **GRANT:**
 - Este comando é usado para conceder permissões a um usuário no MySQL.
2. **ALL PRIVILEGES:**
 - **ALL PRIVILEGES** concede **todas as permissões** disponíveis. Isso inclui permissões como **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **CREATE**, **DROP**, **ALTER**, **INDEX**, etc., além de permissões administrativas como **SHUTDOWN**, **PROCESS**, **RELOAD**, entre outras.
3. **ON *.*:**
 - O ***.*** significa que as permissões são concedidas **para todos os bancos de dados** (* antes do ponto) e **para todas as tabelas** dentro desses bancos de dados (* após o ponto).
4. **TO 'fulano_de_tal'@'127.0.0.1':**
 - **'fulano_de_tal'** é o nome do **usuário** ao qual as permissões estão sendo concedidas.
 - **'127.0.0.1'** indica que o usuário só poderá se conectar ao MySQL a partir do endereço IP **127.0.0.1**.
 - Se você quisesse que ele se conectasse de qualquer lugar, poderia usar **%** (o curinga para "qualquer host").
5. **WITH GRANT OPTION:**
 - **WITH GRANT OPTION** dá ao usuário **a capacidade de conceder** as permissões que ele possui para outros usuários.
 - Isso significa que o usuário **'fulano_de_tal'** pode, por exemplo, conceder suas permissões (como **ALL PRIVILEGES**) a outros usuários. Sem esse **GRANT OPTION**, o usuário não poderia transferir permissões para outros.

Como afeta o acesso?

Acesso Fulano



Acesso Beltrano



MySQL Connections + ↻

Aula

root

127.0.0.1:3306

teste_fulano

fulano_de_tal

127.0.0.1:3306

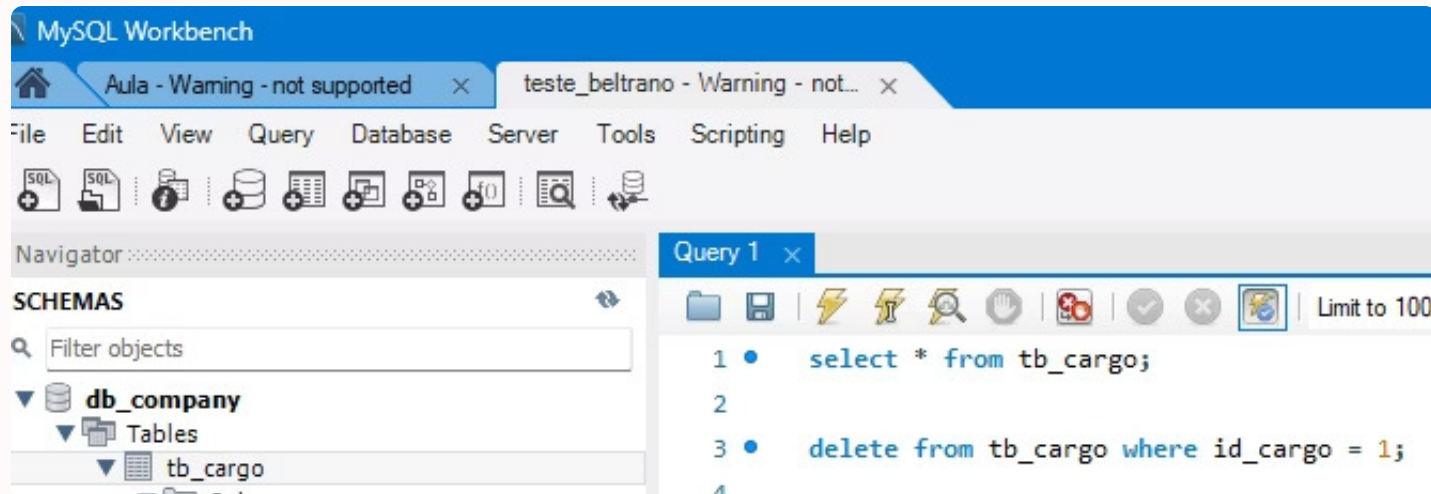
teste_beltrano

beltrano_de_tal

127.0.0.1:3306

Concedendo privilégios específicos

```
grant select on db_company.* to 'beltrano_de_tal'@'127.0.0.1';
```



Error Code: 1142. DELETE **command denied** to user
'beltrano_de_tal'@'localhost' for
table `db_company.tb_cargo`

Alterando Permissões

Antes:

```
grant select on db_company.* to 'beltrano_de_tal'@'127.0.0.1';
```

Atualizando:

```
grant select,delete on db_company.* to 'beltrano_de_tal'@'127.0.0.1';
```

Mostrando os privilégios:

```
SHOW GRANTS FOR 'beltrano_de_tal'@'127.0.0.1';
```

Observação: O `USAGE` é um **privilégio nulo**, ou seja, é o equivalente a: “o usuário existe, mas não tem permissão para fazer absolutamente nada”.

Veja mais em: <https://dev.mysql.com/doc/refman/8.0/en/>

Revogando Permissões (REVOKE)

```
revoke ALL privileges, grant option from 'fulano_de_tal'@'127.0.0.1';
```

Explicando por partes:

1. REVOKE:

- O comando **REVOKE** é usado para **revogar ou remover permissões** que foram anteriormente concedidas a um usuário. Ao contrário do **GRANT**, que concede permissões, o **REVOKE** retira as permissões de um usuário no MySQL.

2. ALL PRIVILEGES:

- A palavra **ALL PRIVILEGES** refere-se a todas as permissões concedidas ao usuário. Isso inclui permissões para fazer **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **CREATE**, **DROP**, entre outras.
- Ao usar **ALL PRIVILEGES**, você está retirando **todas** as permissões que foram concedidas ao usuário **no nível do banco de dados ou tabela especificada**.

3. GRANT OPTION:

- O **GRANT OPTION** refere-se à permissão de **conceder permissões a outros usuários**. Ou seja, quando um usuário tem o **GRANT OPTION**, ele pode repassar as permissões que possui a outros usuários.
- Revogar o **GRANT OPTION** significa que o usuário **não poderá mais conceder permissões a outros**.
- Item obrigatório no Maria DB (O MySQL foi desenvolvido por Monty Widenius, e o Maria DB - altamente compatível com MySQL - também por Monty Widenius, após a compra do MySQL pela Oracle)

No MySQL, quando você **revoga uma permissão** de um usuário, essa mudança **não afeta imediatamente** as sessões que já estão ativas. Ou seja, o usuário ainda terá as permissões revogadas durante a sessão atual até que ele faça um **novo login**.



Dica de Segurança

Evite dar GRANT ALL PRIVILEGES para todos os usuários. Use o princípio do menor privilégio.

O princípio do menor privilégio sugere que cada usuário deve ter apenas as permissões mínimas necessárias para realizar suas funções, reduzindo assim os riscos de segurança.

Transações (START TRANSACTION, COMMIT, ROLLBACK)

Transações garantem que várias operações SQL sejam executadas de forma atômica, ou seja, todas elas devem ocorrer com sucesso ou nenhuma deve ser aplicada. Elas são fundamentais para manter a coerência dos dados.



Exemplo Prático de Transações

Venda de Ingresso
Inserir registro na tabela de vendas

Reversão
Rollback se alguma etapa falhar



Atualização de Status
Marcar ingresso como indisponível

Confirmação
Commit da transação se tudo ocorrer bem



O que é uma Transação?

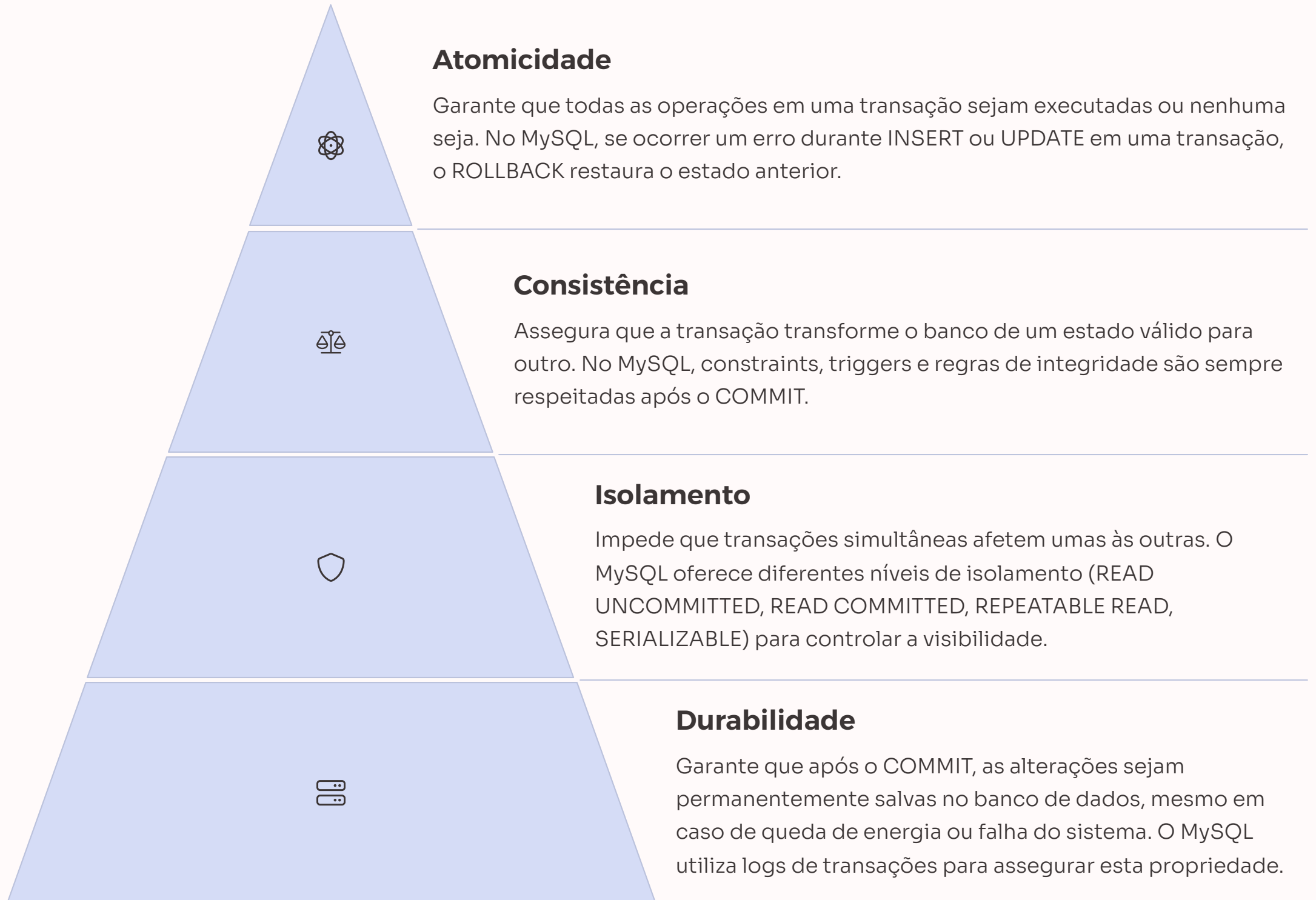
Definição

Uma transação é um bloco de operações SQL agrupadas que devem ser executadas como uma unidade lógica. Ou tudo é concluído com sucesso, ou nada é aplicado ao banco de dados.

Importância

Transações são essenciais para manter a integridade dos dados em operações que envolvem múltiplas etapas interdependentes.

Características ACID



Comandos de Transação no MySQL

Comando	Função
START TRANSACTION	Inicia uma nova transação
COMMIT	Confirma todas as operações realizadas desde o início da transação
ROLLBACK	Desfaz todas as alterações feitas durante a transação
SAVEPOINT nome	Define um ponto de salvamento para rolar parcialmente (avançado)
ROLLBACK TO nome	Retorna ao ponto definido por SAVEPOINT (avançado)



Exemplo de Transação: Compra de Ingresso

```
create user 'ana_maria'@'127.0.0.1' IDENTIFIED BY 'senha123';
```

```
create user 'antonio_carlos'@'127.0.0.1' IDENTIFIED BY 'senha456';
```

```
GRANT ALL PRIVILEGES ON VendaIngressosDB.* TO 'ana_maria'@'127.0.0.1';
```

```
GRANT ALL PRIVILEGES ON VendaIngressosDB.* TO 'antonio_carlos'@'127.0.0.1';
```

Ambiente da Ana Maria

```
START TRANSACTION;

-- Etapa 1: Inserir venda

INSERT INTO Vendas
(id_usuario, id_ingresso, data_venda, quantidade, total)
VALUES (4, 1, NOW(), 1, 850.00);

-- Etapa 2: Tornar ingresso indisponível

UPDATE Ingressos
SET disponivel = FALSE WHERE id_ingresso = 1;

COMMIT;
```

Ambiente do Antonio Carlos

```
START TRANSACTION;

-- Etapa 1: Inserir venda

INSERT INTO Vendas
(id_usuario, id_ingresso, data_venda, quantidade, total)
VALUES (4, 1, NOW(), 1, 850.00);

-- Etapa 2: Tornar ingresso indisponível

UPDATE Ingressos SET disponivel = FALSE WHERE
id_ingresso = 1;

COMMIT;
```

Error Code: 2013. Lost connection to MySQL server during query

Se necessártio:

```
SHOW PROCESSLIST; -- Identifique os IDs com "Sleep" ou "Query" por muito tempo
```

```
KILL 37;
```

Exemplo com ROLLBACK e Savepoint

Imagine que você inicia uma transação para registrar uma venda e tornar o ingresso indisponível. No meio do caminho, algo dá errado (ou você decide rever uma etapa), e não quer perder **tudo**, mas apenas **parte** da transação.

Usamos `SAVEPOINT` para marcar pontos seguros, e `ROLLBACK TO` para voltar a um ponto específico.

START TRANSACTION;

-- Etapa 1: Inserir venda

INSERT INTO Vendas (id_usuario, id_ingresso, data_venda, quantidade, total) VALUES (4, 1, NOW(), 1, 850.00);

-- Criar um ponto de recuperação após o INSERT

SAVEPOINT depois_da_venda;

-- Etapa 2: Tornar ingresso indisponível

UPDATE Ingressos SET disponivel = FALSE WHERE id_ingresso = 1;

-- Suponha que você percebe um erro aqui: quer cancelar a atualização, mas manter a venda

ROLLBACK TO depois_da_venda;

-- Decide manter a venda, e COMMIT a transação sem atualizar o ingresso

COMMIT;

Mais Perguntas para Discussão



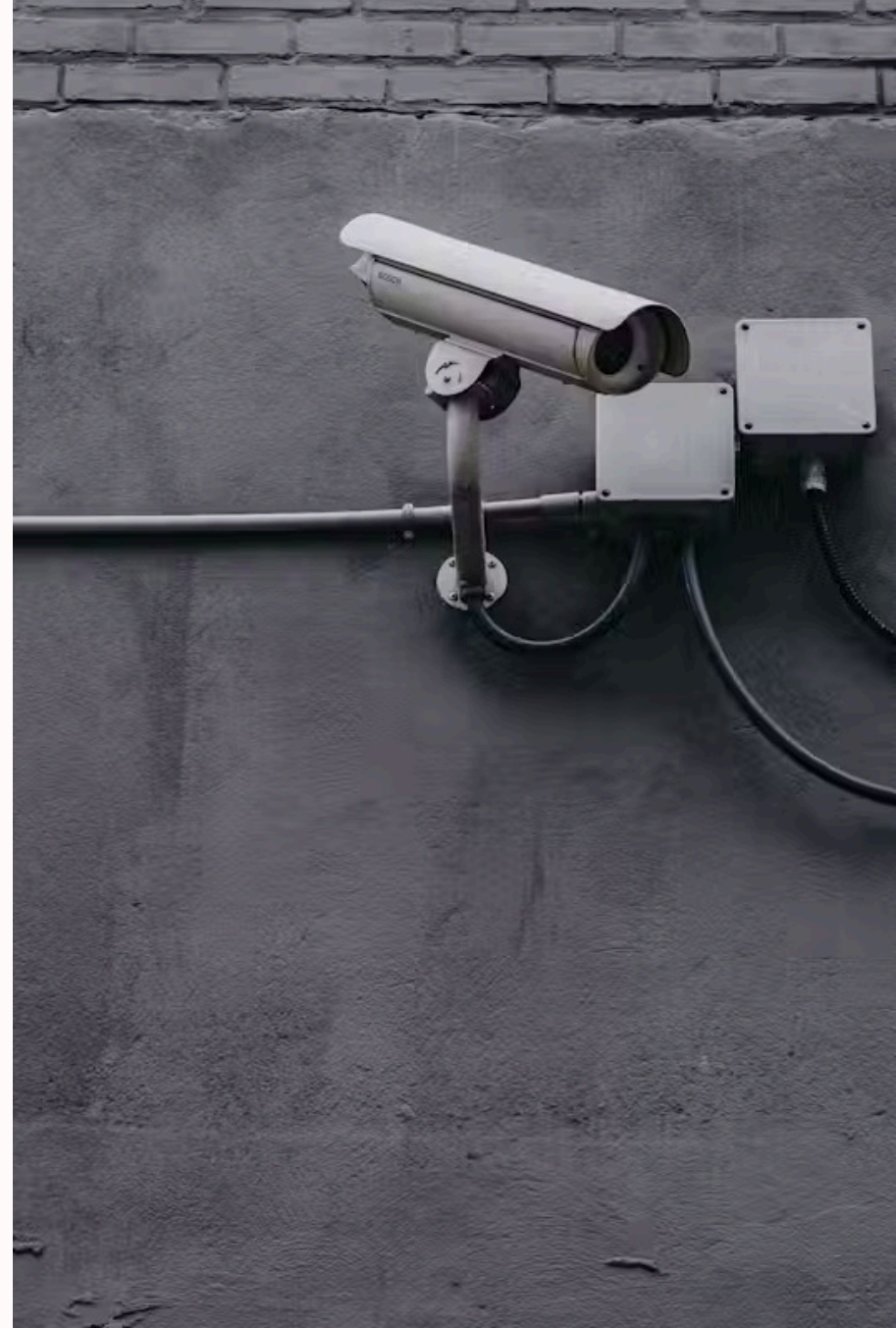
Princípio do menor privilégio

Por que é importante aplicar o princípio do menor privilégio ao conceder permissões?



Isolamento de transações

Em sistemas com múltiplos usuários acessando simultaneamente, o que poderia acontecer se não houvesse isolamento de transações?



Riscos de Permissões Excessivas



Vazamento de dados

Acesso indevido a informações sensíveis



Alterações não autorizadas

Modificações maliciosas ou acidentais



Exclusão de dados

Perda permanente de informações importantes



Benefícios das Transações



Integridade dos dados

Garante que os dados permaneçam consistentes mesmo em caso de falhas



Operações atômicas

Assegura que operações relacionadas sejam tratadas como uma unidade



Concorrência segura

Permite que múltiplos usuários trabalhem simultaneamente sem conflitos



Recuperação de erros

Facilita a reversão de operações em caso de problemas

Cenários de Uso de Transações



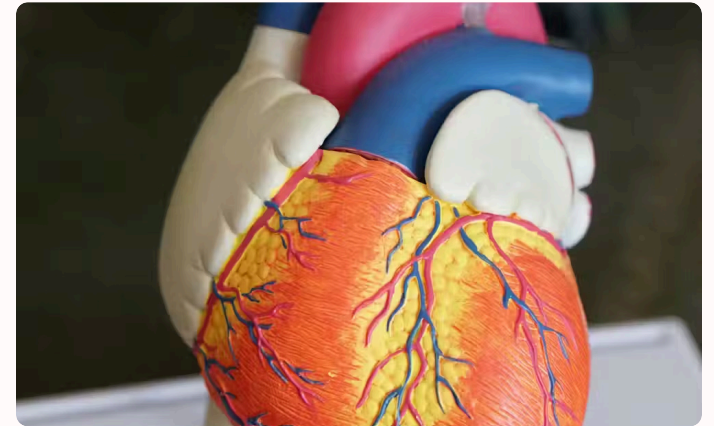
Transferências bancárias

Debitar de uma conta e creditar em outra deve ocorrer como uma única operação atômica



Compras online

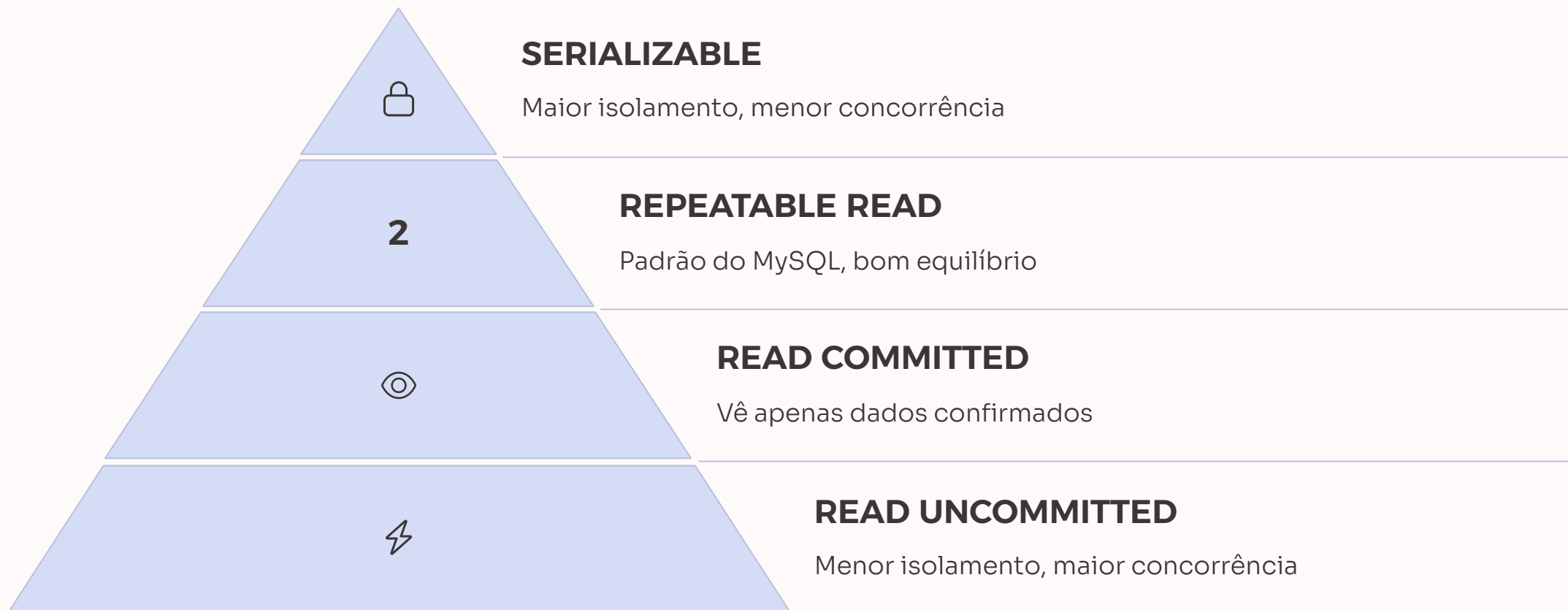
Registrar pedido, processar pagamento e atualizar estoque devem ser concluídos juntos

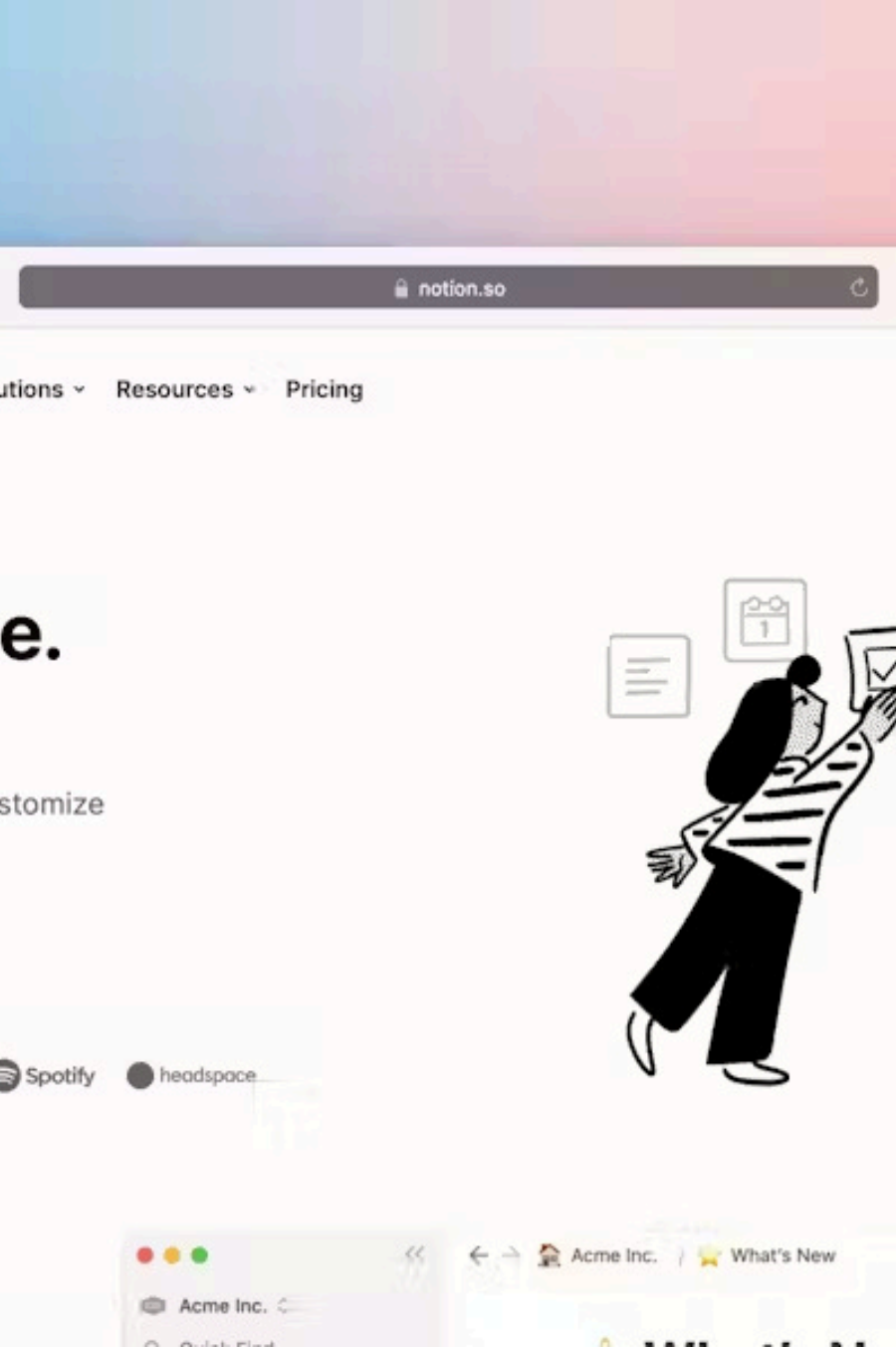


Sistemas de reservas

Verificar disponibilidade e confirmar reserva precisam ser tratados como uma unidade

Níveis de Isolamento de Transações





Problemas de Concorrência

Leituras sujas (Dirty Reads)

Ocorre quando uma transação lê dados que foram modificados por outra transação que ainda não foi confirmada (committed).

Leituras não repetíveis (Non-repeatable Reads)

Acontece quando uma transação lê o mesmo registro duas vezes e obtém valores diferentes porque outra transação modificou o registro entre as leituras.

Leituras fantasmas (Phantom Reads)

Ocorre quando uma transação executa a mesma consulta duas vezes e a segunda consulta retorna linhas adicionais que foram inseridas por outra transação.

Melhores Práticas para Transações



Mantenha transações curtas

Transações longas aumentam o risco de bloqueios e conflitos



Trate erros adequadamente

Sempre inclua tratamento de exceções com ROLLBACK em caso de falhas



Agrupe operações relacionadas

Inclua apenas operações que precisam ser atômicas na mesma transação



Escolha o nível de isolamento apropriado

Balance segurança e desempenho conforme as necessidades da aplicação



Gerenciando Usuários no MySQL

Criar usuário

```
CREATE USER 'nome_usuario'@'host' IDENTIFIED BY 'senha';
```

Conceder permissões

```
GRANT tipo_permissao ON banco.tabela TO 'nome_usuario'@'host';
```

Verificar permissões

```
SHOW GRANTS FOR 'nome_usuario'@'host';
```

Revogar permissões

```
REVOKE tipo_permissao ON banco.tabela FROM 'nome_usuario'@'host';
```

Remover usuário

```
DROP USER 'nome_usuario'@'host';
```

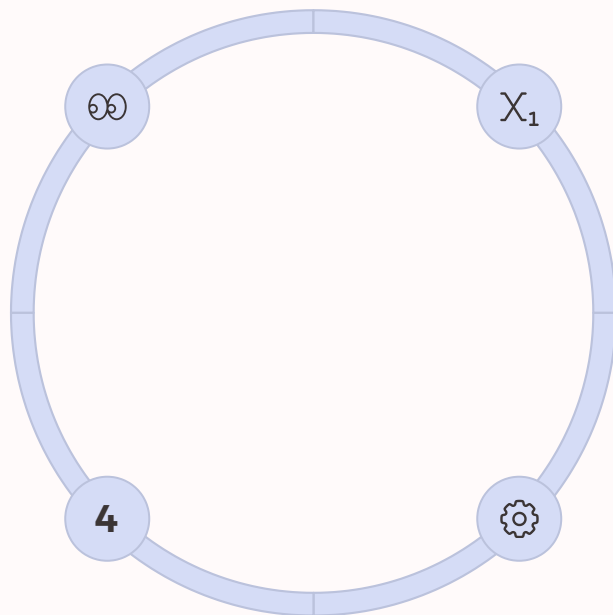
Tipos de Permissões no MySQL

Permissões de Leitura

- SELECT
- SHOW VIEW

Permissões Administrativas

- GRANT OPTION
- SUPER
- PROCESS



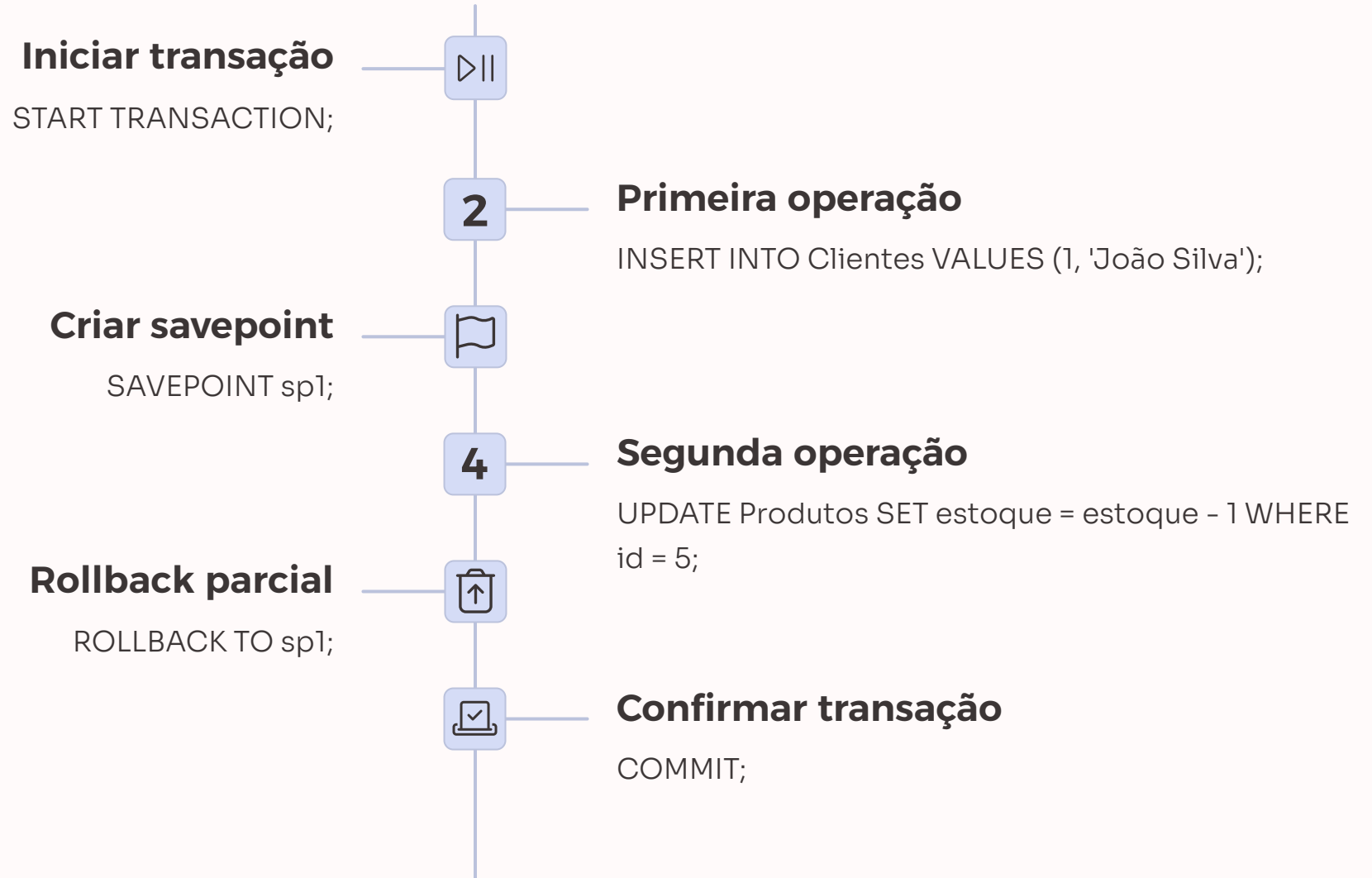
Permissões de Escrita

- INSERT
- UPDATE
- DELETE

Permissões de Estrutura

- CREATE
- ALTER
- DROP

Savepoints em Transações



Impacto da Ausência de Transações

Cenário: Sistema de Vendas de Ingressos

Sem transações, as seguintes inconsistências podem ocorrer:

- Venda registrada, mas ingresso ainda disponível
- Ingresso marcado como vendido, mas venda não registrada
- Pagamento processado, mas venda não finalizada
- Mesmo ingresso vendido para dois clientes diferentes

Consequências

- Prejuízos financeiros
- Clientes insatisfeitos
- Overbooking de eventos
- Dados inconsistentes para relatórios
- Dificuldade em auditorias
- Problemas legais e de conformidade



Monitoramento de Transações

5.2s

Tempo médio de transação

Duração média das transações no sistema

0.3%

Taxa de rollback

Percentual de transações que precisaram ser revertidas

1250

Transações por minuto

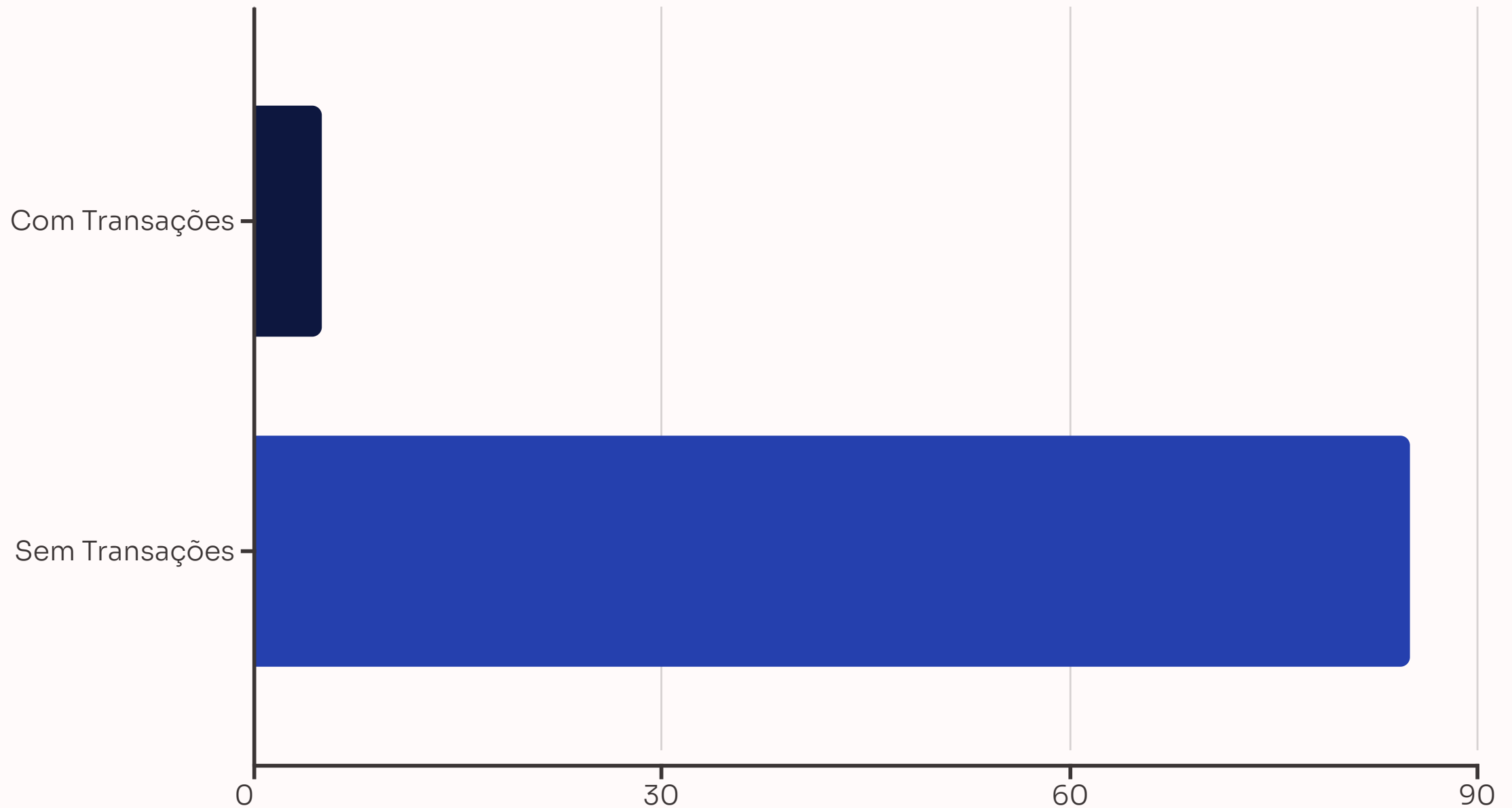
Volume de transações processadas

99.9%

Disponibilidade

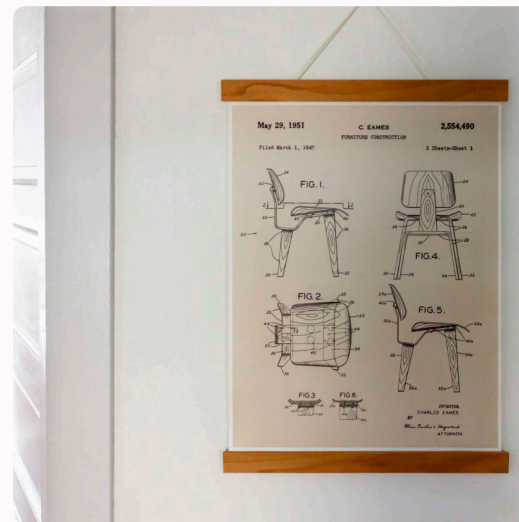
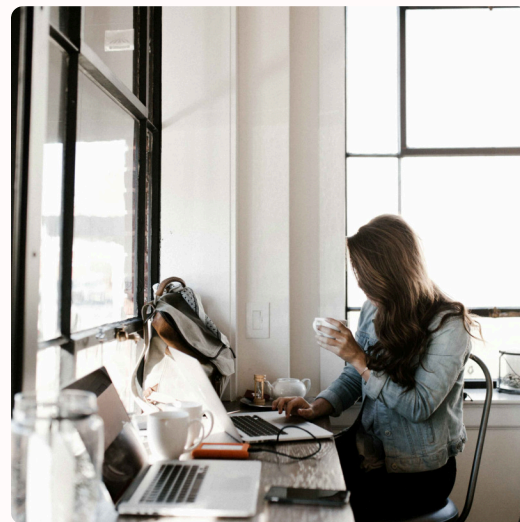
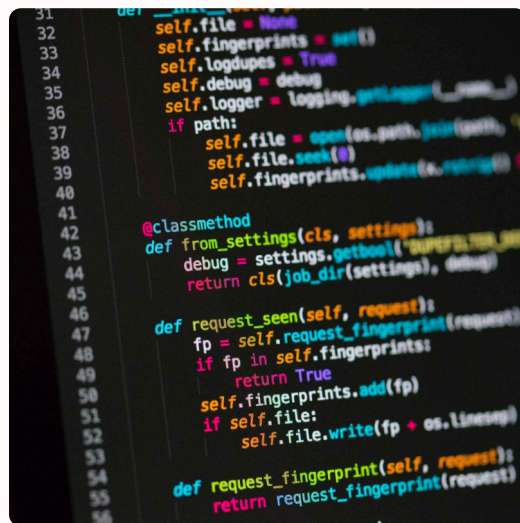
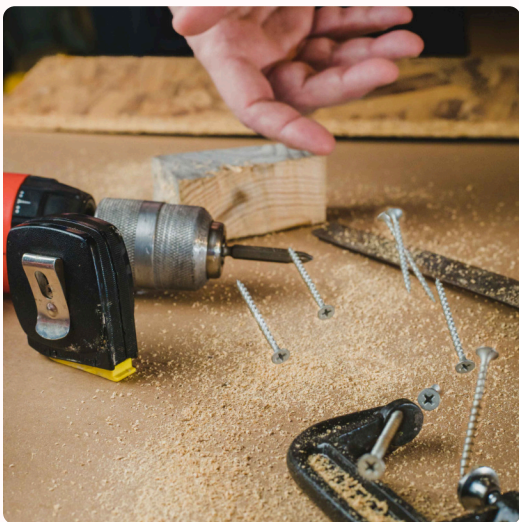
Tempo de atividade do banco de dados

Comparação: Com e Sem Transações



O gráfico demonstra claramente como o uso de transações reduz drasticamente o risco de inconsistências no banco de dados, especialmente em operações que envolvem múltiplas tabelas ou etapas.

Exercício Prático de Transações



Implemente uma transação que registre a venda de múltiplos ingressos para um mesmo cliente, atualizando o estoque e registrando o pagamento. Inclua tratamento de erros com ROLLBACK em caso de falhas.

Resumo: Permissões e Transações

Permissões

- Controlam o acesso aos dados
- Utilizam GRANT e REVOKE
- Seguem o princípio do menor privilégio
- Protegem contra acessos indevidos
- Garantem conformidade e segurança

Transações

- Garantem operações atômicas
- Mantêm a consistência dos dados
- Utilizam START TRANSACTION, COMMIT e ROLLBACK
- Seguem propriedades ACID
- Previnem inconsistências em operações complexas